

Design of Low-power YOLO Accelerator Based on FPGA

Yunhui Qu^{1,2,a}, Jintao Wang^{1,2,*}

¹School of Information and Communication Engineering, Communication University of China, Beijing, China

²Key Laboratory of Media Audio & Video, Ministry of Education, Beijing, China

^a18838067252@163.com, *Corresponding Author: wangjt@cuc.edu.cn

Abstract—In order to reduce the power consumption and hardware resource consumption of deploying YOLO algorithms on embedded devices, we designed a low-power YOLOv4-Tiny algorithm accelerator based on FPGA. Firstly, we introduced an attention mechanism into the FPN module of the YOLOv4-Tiny algorithm and effectively trained the improved model on a computer platform, utilizing 16-bit fixed-point quantization and layer fusion to reduce the model's computational load. Secondly, based on the FPGA hardware platform, we maximized the use of techniques such as pipelining, loop unrolling, and ping-pong operations to design hardware accelerators for speeding up computationally intensive units such as convolutional and pooling layers in the YOLOv4-Tiny algorithm. Finally, we completed the hardware system design for the entire algorithm. The experimental results show that our improved algorithm improves the mean Average Precision (map) by 1.9% on the MS COCO dataset. When the clock frequency of FPGA development board is 150 MHz, the total power consumption of our improved algorithm in the hardware system is only 3.415W. At the same time, the performance per watt reaches 6.76 GOPS/W, which is 23.3 times of the CPU platform and 10.9 times of the GPU platform of the same algorithm. The system can significantly reduce hardware resource consumption and power consumption.

Keywords—FPGA, Object Detection, YOLO accelerator, low power consumption

I. INTRODUCTION

With the rise of deep learning, object detection plays a vital role in computer vision applications such as security monitoring [1], authentication [2], unmanned driving [3], intelligent security [4], and industrial automation [5]. Early target detection mostly uses traditional methods for image feature extraction, such as Viola-Jones [6], HOG [7], DPM [8] and so on. These methods are slow, low accuracy, and poor performance across domain. Deep learning was first proposed by Hinton et al. [9] in 2006, which pushed target detection to artificial intelligence and accelerated the extraction of image features. The target detection algorithm based on CNN can automatically learn and discover various features of the object to be detected under a large amount of data, improve the accuracy and detection speed of the detection results, and finally achieve better results than the traditional algorithm. In recent years, traditional general-purpose processors such as CPU and GPU have the characteristics of low performance, high power consumption, and high price when processing CNN. FPGA has the advantages of low power consumption, high hardware flexibility, and parallel computing capabilities. It is a good choice for achieving target detection algorithms and is now widely used to implement CNN acceleration [10].

YOLO is a common algorithm in the field of object detection. The original YOLO [11] was written by Joseph Redmond and Ali Farhadi in the Darknet custom framework. YOLOv1 is the first object detection network that combines the problem of drawing bounding boxes and identifying class labels in an end-to-end differentiable network. YOLOv2 [12] introduced some improvements, including the use of finer grids, multi-scale prediction, and Anchor Boxes to improve positioning accuracy. YOLOv3 [13] introduces the multi-scale feature Darknet-53 backbone network and uses three different scale detection layers to increase the detection accuracy of small targets. YOLOv4 [14] has significantly improved the accuracy and speed of object detection by introducing CSPNet and a large number of technological innovations, making it more outstanding in complex scenarios. YOLOv4-Tiny improves inference speed by streamlining YOLOv4 structure and reducing its network size.

In this paper, we propose an improved YOLOv4-Tiny algorithm and deploy it on the low-power hardware platform FPGA. In the hardware acceleration design process of YOLO model, we use a variety of optimization methods, such as multi-channel input and output, data dynamic fixed-point quantization, BN layer operation optimization and so on. After that, it focuses on the design of the IP core of the convolution layer and the pooling layer in the improved YOLOv4-Tiny algorithm.

II. DEEP LEARNING NETWORK MODEL

A. Yolov4-Tiny Network Model

YOLOv4-Tiny is a lightweight model of YOLOv4. Compared with YOLOv4, the detection accuracy is slightly lower, but the detection speed is significantly improved. The number of parameters is only 1/10 of YOLOv4, which reduces the hardware requirements. Compared with other algorithms, the YOLOv4-Tiny algorithm can maintain high detection accuracy with a relatively lightweight network structure and a low number of parameters, so it is more suitable as a target detection algorithm deployed on hardware for reasoning.

YOLOv4-Tiny is composed of CSPDarknet53-tiny, FPN and two YOLO Heads. CSPDarknet53-tiny is mainly composed of CLB and CSP. CLB consists of convolutional layer, BN layer and LeakyReLU. CSP is used to strengthen the construction of FPN and solve the problem of model degradation caused by increasing network depth. The main function of FPN is to perform multi-scale feature fusion, so that the network can effectively detect and understand targets at different scales. Two YOLO Heads correspond to the

detection tasks of large and small targets, respectively. The YOLOv4-Tiny network structure is shown in Figure 1.

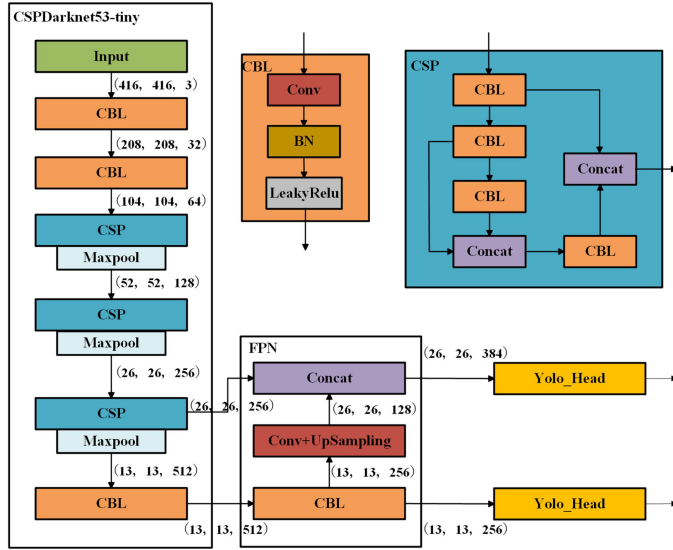


Fig. 1. YOLOv4-Tiny model structure.

B. SE Attention Mechanism

In the feature transfer process of YOLOv4-Tiny network, there is a lack of mechanism to automatically filter important information, which may ignore some important information in detection. In order to solve this problem, we introduce the SE [15] attention mechanism. The SE attention mechanism enables the model to selectively focus on different parts of the input when processing the input data, rather than blindly average all the information, which helps to improve the model's ability to process important information. The SE attention mechanism module mainly includes both Squeeze and Excitation. The network structure is shown in Figure 2.

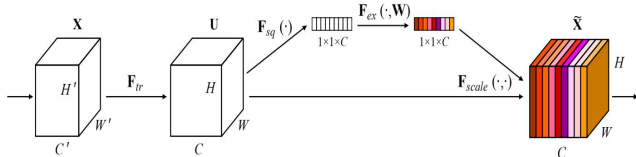


Fig. 2. SE attention mechanism.

Squeeze: The input feature map is pooled by global average, and a $1 \times 1 \times C$ feature map is obtained after pooling. The formula is shown in (1), where u_c denotes the eigenvector.

$$z_c = F_{sq}(u_c) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W u_c(i, j) \quad (1)$$

Excitation: The feature map after Squeeze is renormalized by two fully connected layers and activation functions. In order to facilitate the deployment in the FPGA, the 1×1 convolution is used instead of the full connection layer. The formula is shown in (2), σ represents the Sigmoid activation function, and g represents the ReLU function.

$$s = F_{ex}(z, W) = \sigma(g(z, W)) \quad (2)$$

Finally, the Scale operation is performed, which is to multiply the learned activation value s_c of each channel and the original eigenvalue u_c to obtain the weight coefficient of each channel. The formula is shown in (3).

$$F_{scale}(s_c, u_c) = s_c \times u_c \quad (3)$$

C. Improved YOLOv4-Tiny Network Structure

SE is a lightweight plug-and-play module. In order to preserve the initial features extracted by the backbone network, the SE attention mechanism is integrated into the FPN module. With the introduction of the SE attention mechanism, the network becomes more focused on crucial information within the feature maps. It also has the capability to suppress interference from irrelevant and secondary information, thereby enhancing the detection accuracy of the network model. The improved network model is illustrated in Figure 3.

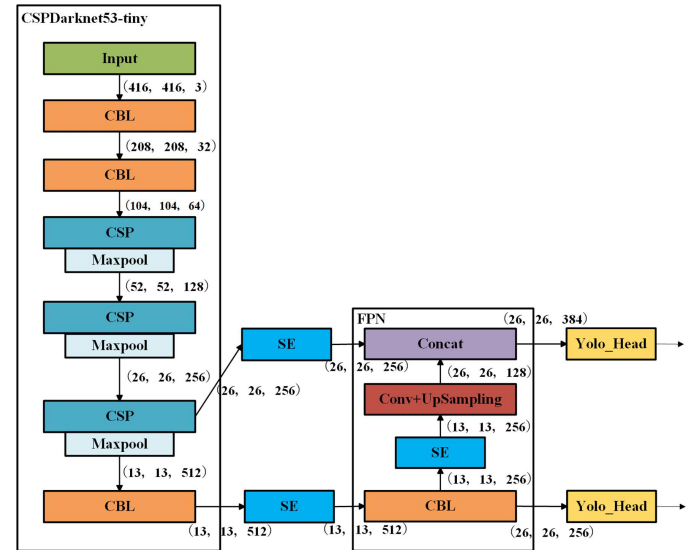


Fig. 3. Improved YOLOv4-Tiny model structure.

III. HARDWARE ACCELERATION OPTIMIZATION DESIGN OF THE YOLO MODEL

In the YOLO model, operations between each layer are basically executed sequentially, and the data between two adjacent layers belongs to the input-output relationship. In order to increase the data processing speed while minimizing the computational burden of the hardware accelerator, some optimization strategies need to be adopted during the hardware accelerator design stage.

A. Multi-channel Input and Output

Based on the characteristics of the FPGA internal interfaces, we make full use of interface bandwidth resources by selecting four Slave AXI_HP interfaces as channels for data transfer between DDR memory and the PL side. By setting different address offsets in memory, the data to be transferred is divided into four portions and transmitted simultaneously through the four Slave AXI_HP interfaces. This enhances the data throughput between DDR and the PL side, reducing time losses associated with data movement. In the Block Design section of Vivado 2019.2 design tool, the configuration of the four Slave AXI_HP interfaces is illustrated in Figure 4

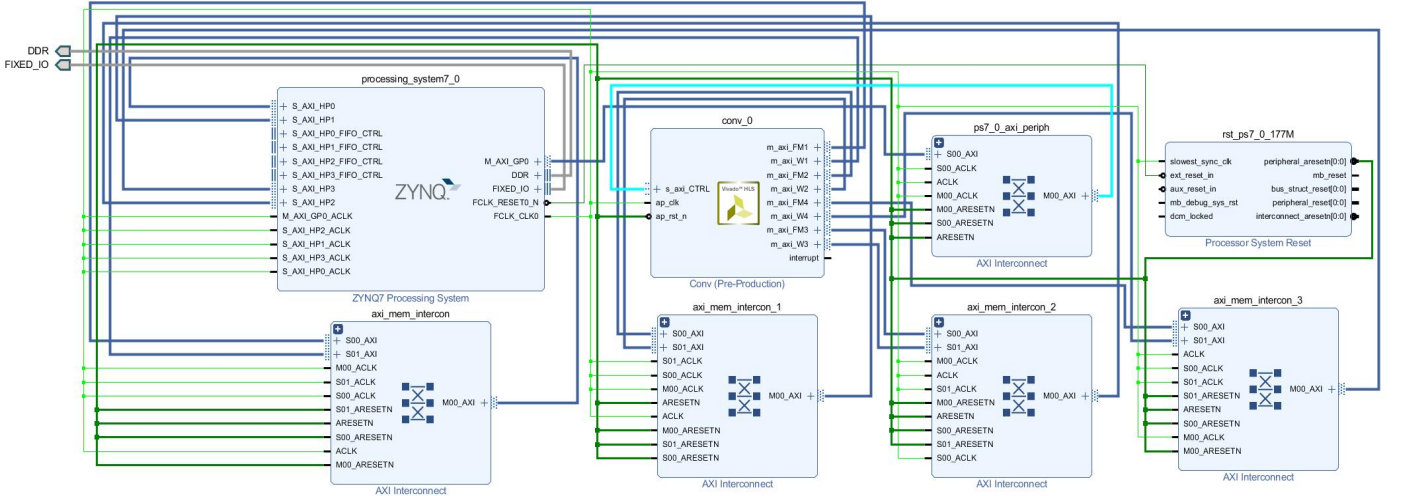


Fig. 4. Diagram of 4-channel interface

B. Data Fixed-point Quantization

Convolution operation is composed of numerous multiplication and addition operations, with the precision of the data involved leading to varying resource consumption on FPGA. The primary resources consumed are DSP and LUT [16]. The specific resource consumption is outlined in Table I.

TABLE I. COMPARISON OF RESOURCE CONSUMPTION

Operation Types	DSP	LUT
Addition(32 bit floating point)	2	214
Multiplication(32 bit floating point)	3	135
Addition(16 bit fixed point)	0	47
Multiplication(16 bit fixed point)	1	101

From Table I, it is evident that the resource consumption for addition and multiplication operations significantly decreases when transitioning from 32-bit floating-point numbers to 16-bit fixed-point numbers. Therefore, we quantize the model's 32-bit floating-point parameters into 16-bit fixed-point parameters. This transformation shifts the algorithm's floating-point data operations to fixed-point multiply-add operations, making it more suitable for FPGA platform computation and reducing the utilization of computational resources. The decimal mathematical expression for fixed-point numbers is illustrated in Equation (4), where bw is the bit width of the fixed-point number, f_i is the length of the fractional part, B_i represents the quantization precision, B_i is between 0 and the quantization base, and the most significant bit denotes the sign.

$$V_{int} = \sum_{i=0}^{bw-1} 2^{-f_i} \times B_i \times 2^i \quad (4)$$

C. BN Layer Operation Optimization

The BN layer accelerates the convergence speed of the network and enhances its generalization ability. During the network inference process, to speed up the inference speed, reduce the network's parameter and computation overhead, it is

common to fuse the convolutional layer with the BN layer. The principle of layer fusion is as follows.

$$Conv(X) = X \times W + B \quad (5)$$

$$BatchNorm(x) = ((x - \mu) / \sqrt{\sigma^2 + \epsilon}) \times \gamma + \beta \quad (6)$$

$$\Rightarrow ConvBN(X) = ((X \times W + B - \mu) / \sqrt{\sigma^2 + \epsilon}) \times \gamma + \beta \quad (7)$$

$$\Rightarrow ConvBN(X) = X \times W \times \gamma / \sqrt{\sigma^2 + \epsilon} + (B - \mu) \times \gamma / \sqrt{\sigma^2 + \epsilon} + \beta \quad (8)$$

$$\Rightarrow ConvBN(X) = X \times W' + B' \quad (9)$$

$$W' = W \times \gamma / \sqrt{\sigma^2 + \epsilon}, B' = (B - \mu) \times \gamma / \sqrt{\sigma^2 + \epsilon} + \beta.$$

$Conv(\cdot)$ represents the normalized result, and $BatchNorm(\cdot)$ is the output value after BN layer processing. μ denotes the mean, σ^2 denotes the variance, and ϵ is a protective parameter for the fraction. γ is the scaling factor of the BN layer, and β is the shift parameter of the BN layer; both γ and β are updated during the network training. W and B are the original convolutional weights and biases, while W' and B' are the weights and biases after fusion.

IV. SYSTEM IMPLEMENTATION

A. Overall Program

We employ the XC7Z035 hardware platform, utilizing an ARM+FPGA architecture to accelerate neural network inference. ARM is primarily responsible for task scheduling and data distribution, including the memory control module, data input/output interface, and post-processing for YOLO detection. The FPGA handles parallel computation for modules like convolution and pooling, composed mainly of on-chip memory BRAM and multiple accelerated IP cores. The SD card stores the parameters of the input system's network model and image parameters, saving the results processed by the system back to the SD card. The DDR controller uses the AXI bus interface to cache data from the SD card to DDR, simultaneously facilitating data interactions with other modules and storing all interactive data and final computed results.

BRAM, an on-chip storage device within the FPGA, is responsible for storing input and output data. Given the numerous algorithmic operations throughout the process, to minimize power consumption and achieve minimal latency, it's crucial to leverage the strengths of both the PL and PS within the ZYNQ for algorithm deployment, as illustrated in Figure 5 of the ZYNQ system architecture.

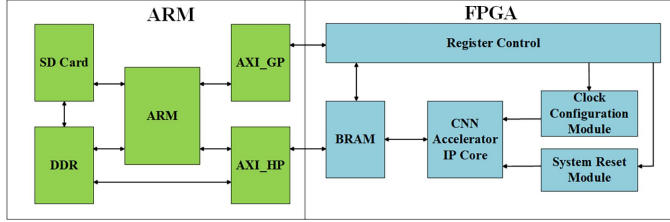


Fig. 5. Overall system structure of FPGA implementation.

B. Convolutional Module Design

In the YOLOv4-Tiny algorithm, the convolution and pooling layers account for more than 90% of the total calculation of the network, and the convolution layer accounts for the largest proportion. Therefore, in the FPGA implementation of the algorithm, the key lies in the design of convolution layer and pooling layer.

1) Design of Convolution IP kernel

The pseudocode for the convolution operation is illustrated in Algorithm 1. Here, T_m represents the number of output feature maps, (T_r, T_c) denotes the size of the output feature maps, T_n is the number of input feature maps, and (N_{kx}, N_{ky}) represents the convolution kernel size. The convolution operation is effectively implemented through four nested loops. However, this approach is inefficient, necessitating optimization for improved performance.

Algorithm 1: Convolutional layer algorithm

```

Input: Fin      //Input Feature Map
weight         //Weight Parameters
Output: Fout    //Output Feature Map
1  for(mm=0;mm<Tm;mm++)
2    for(rr=0;rr<Tr;rr++)
3      for(cc=0;cc<Tc;cc++)
4        for(nn=0;nn<Tn;nn++)
5          for(kx=0;kx<NKx;kx++)
6            for(ky=0;ky<NKy;ky++)
7              Fout[mm][rr][cc] += Fin[nn][rr*stride+kx][cc*stride+ky]
                                     *weight[mm][nn][kx][ky];

```

In order to enhance the processing speed of the system, we have implemented parallel computing across input channels. This involves simultaneous computation of feature maps from multi-channel inputs and corresponding convolutional layers. Additionally, we have optimized input/output operations through parallel and loop unrolling, and employed pipeline processing for the PE (Processing Element) module. Furthermore, two buffers have been introduced after input feature maps and weights, as well as before output feature maps, enabling each module to work concurrently with interleaved read and write operations. This significantly boosts

the system's throughput. The overall architecture of the convolution module is illustrated in Figure 6.

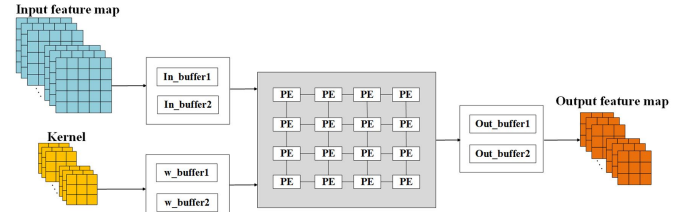


Fig. 6. Convolution layer computation structure.

2) Design of Pooling IP kernel

The main function of the pooling layer is to reduce computation and the number of parameters while effectively retaining crucial features, thereby enhancing computational efficiency. In YOLOv4-Tiny, the maximum pooling has a range of 2×2 with a stride of 2, effectively reducing the input size to half its original. The max pooling operation involves finding the maximum value within the pooling size range. While comparing the four values individually can save resources, it is less efficient. We have optimized the logic by pairwise comparisons of the four data before conducting the final maximum value comparison. The computational process is illustrated in Figure 7.

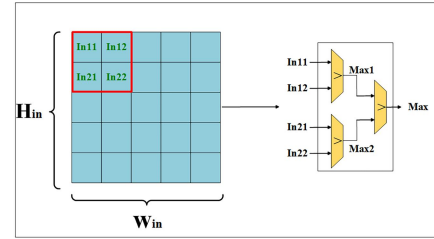


Fig. 7. Pooling computation structure.

In the attention mechanism module, the global average pooling is used. The pseudo-code of the average pooling algorithm implemented in this paper is shown in Algorithm 2, where T_n is the number of input feature maps, (M, N) is the size of the input feature map, and S is the size of the pooling window.

Algorithm2: Average pooling algorithm

```

Input: Fin      //Input Feature Map
Output: Fout    //Output Feature Map
1  for(nn=0;nn<Tn;nn++){
2    for(int i = 0; i < M/S; i++){
3      for(int j = 0; j < N/S; j++){
4        for(int x = 0; x < S; x++){
5          for(int y = 0; y < S; y++){
6            sum += Fin[nn][i*S+x][j*S+y];
7          Fout[nn][i][j] = sum / (S * S);

```

V. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experimental Platform

The board we utilized is the Xilinx ZYNQ series XC7Z035, equipped with a dual-core Cortex-A9 processor, comprising PL

and PS. We designed the accelerator hardware core module using Vivado HLS 2019.2, performed synthesis and placement in Vivado 2019.2, and completed the PS-side design using the Xilinx Vitis IDE 2019.2. Training and validation were conducted using the MS COCO dataset [17].

B. FPGA Resource Consumption Analysis

Table II shows the consumption of FPGA resources in this paper. The consumption of DSP resources is mainly used for the multiplication and addition of convolution function modules. The consumption of BRAM resources is mainly used to implement data caching. The control signal and state configuration on the PS side use FF resources. The multiplexing and transmission of data in the operation process requires logical cooperation, resulting in the consumption of LUT resources.

TABLE II. FPGA RESOURCE CONSUMPTION

Resource	Utilization	Available	Utilization%
LUT	37219	171900	21.65
LUTBAM	7234	70400	10.28
FF	45282	343800	13.17
BRAM	94.50	500	18.90
DSP	224	900	24.89

C. Comparison of Detection Results on Different Platforms

The test results before and after the improvement of the model are compared on the PC platform and the FPGA platform, as shown in Figure 8 and 9. Figure 8 shows the test results before and after the improvement of YOLOv4-Tiny network on PC platform. It can be seen that the improved network recognition effect is better. Figure 9 shows the test results before and after the improvement of the YOLOv4-Tiny network on the FPGA platform. It can be seen that the improved network recognition accuracy has been significantly improved. The recognition accuracy of the FPGA platform is slightly lower than that of the PC platform. This is because the 16-bit fixed-point number is used to replace the 32-bit floating-point number of the original PC platform to reduce the recognition accuracy.

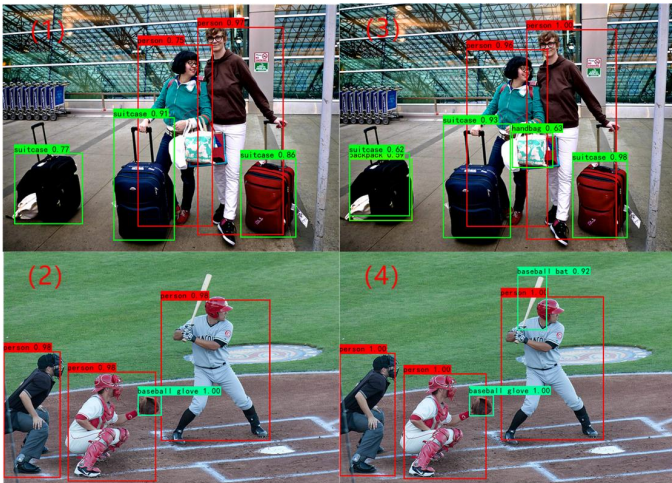


Fig. 8. Object detection on the PC platform using YOLOv4-Tiny (1-2) and our improved method (3-4).

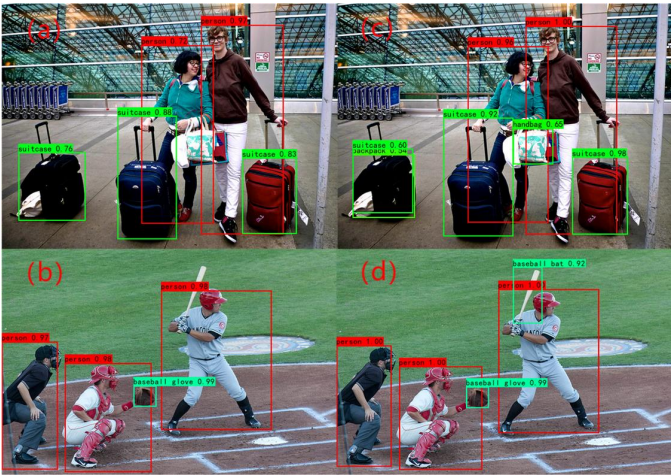


Fig. 9. Object detection on the FPGA platform using YOLOv4-Tiny (a-b) and our improved method (c-d).

D. Performance Analysis

As shown in Table III, the low-power YOLO accelerator we designed based on FPGA achieves a frame rate of 3.32 FPS, which is 2.22 times that of the CPU platform. The power consumption of the CPU is 13.18 times that of the FPGA platform. Despite a significant difference in FPS compared to the GPU, the GPU power consumption is approximately 117.13 times that of the FPGA platform. Implementing the Yolov4-Tiny algorithm on the FPGA platform effectively reduces the system's power consumption. Although the GOPS of the FPGA platform is lower than that of the CPU and GPU, the power consumption of the FPGA platform is lower. The FPGA platform achieves a higher energy efficiency ratio, which is 23.31 times and 10.90 times that of the CPU and GPU respectively.

TABLE III. COMPARISON OF DIFFERENT IMPLEMENTATION PLATFORMS

Platform	CPU	GPU	FPGA
Device	Intel(R)Core(TM) i5-7300HQ	NVIDIA A800 SXM4 80G	ZYNQ7035 (This Work)
Clock	2.50GHz	1.155GHz	150MHZ
FPS	1.93	34.84	3.32
Power	45W	400W	3.415W
GOPS	13.42	248.23	23.09
Energy efficiency ratio (GOPS/w)	0.29	0.62	6.76

TABLE IV. PERFORMANCE COMPARISON BETWEEN THIS PAPER AND FPGA IMPLEMENTATION OF OTHER YOLO SERIES ALGORITHMS

	YOLOv3-Tiny[18]	YOLOv4-Tiny[19]	This Paper
Platform	Zedboard	Virtex UltraScale+ VCU118	ZYNQ7035
FPS	1.88	3.33	3.32
Power	3.36W	19.5W	3.415W
GOPS	10.45	—	23.09
GOPS/w	3.11	—	6.76

At present, there are still many researches on implementing YOLO series algorithms on FPGA platform. The results of this

paper are compared with some literatures, as shown in Table IV. Compared with Yu [18], this paper has higher frame rate, greater throughput, and higher energy efficiency ratio. The author of Eid [19] also implemented the FPGA implementation of the YOLOV4-Tiny algorithm. Although the FPS is slightly higher than this article, its power consumption is 5.71 times that of this article.

VI. CONCLUSION

In this paper, we design a low-power YOLO accelerator based on FPGA. By enhancing the YOLOv4-Tiny algorithm, we improved the detection accuracy. Prior to deployment, we conducted 16-bit fixed-point quantization and layer fusion on the network model parameters to reduce the computational complexity of hardware inference. At the same time, we use the parallel characteristics of FPGA to accelerate the convolutional neural network, which greatly improves the speed of image recognition. At a working frequency of 150 MHz, the power consumption of this design is 3.415 W, and the recognition time for a single image is approximately 301ms. The speed is 2.22 times faster than a typical CPU, with power consumption being only 7.58% of a regular CPU. Experimental results demonstrate that this design exhibits characteristics of low power consumption and high performance, holding significant practical value in edge computing or embedded devices.

REFERENCES

- [1] H. Zhang et al., "Correlation and binding technology of passenger security check information based on video target detection and tracking," 2022 IEEE 4th International Conference on Civil Aviation Safety and Information Technology (ICCASIT), Dali, China, 2022, pp. 599-604, doi: 10.1109/ICCASIT55263.2022.9986752.
- [2] K. A. O. V. Kodikara, P. Hettiarachchi, D. M. J. Prathapa, J. M. A. M. S. Jayakody and P. S. Haddela, "Surveillance based Child Kidnap Detection and Prevention Assistance," 2022 IEEE 7th International conference for Convergence in Technology (I2CT), Mumbai, India, 2022, pp. 1-5, doi: 10.1109/I2CT54291.2022.9824093.
- [3] T. Chiang, Q. Li and M. Huang, "Object Detection and Segmentation Method for Unmanned Vehicles in Bird's-Eye View Based on Peripheral Camera," 2022 2nd International Conference on Algorithms, High Performance Computing and Artificial Intelligence (AHPCAI), Guangzhou, China, 2022, pp. 578-581, doi: 10.1109/AHPCAI57455.2022.10087610.
- [4] Y. Wang, Z. Ma, H. Fang, C. Hu, Y. Cao and D. He, "Pedestrian Recognition System for Smart Security Robot using Pedestrian Re-identification Algorithm," 2021 International Conference on Computer Engineering and Artificial Intelligence (ICCEAI), Shanghai, China, 2021, pp. 371-376, doi: 10.1109/ICCEAI52939.2021.00074.
- [5] W. Deng, C. Zhang, Z. Zou, M. Gao, X. Wang and W. Yu, "Peg-in-hole assembly of industrial robots based on Object detection and Admittance force control," 2021 36th Youth Academic Annual Conference of Chinese Association of Automation (YAC), Nanchang, China, 2021, pp. 672-677, doi: 10.1109/YAC53711.2021.9486619.
- [6] Paul Viola and Michael J. Jones. 2004. Robust Real-Time Face Detection. *Int. J. Comput. Vision* 57, 2 (May 2004), 137-154. <https://doi.org/10.1023/B:VISI.0000013087.49260.fb>.
- [7] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), San Diego, CA, USA, 2005, pp. 886-893 vol. 1, doi: 10.1109/CVPR.2005.177.
- [8] Rother, C., Kolmogorov, V., Blake, A. "GrabCut" interactive foreground extraction using iterated graph cuts[J]. *ACM transactions on graphics (TOG)*, 2004, 23(3): 309-314. <https://doi.org/10.1145/1015706.1015720>.
- [9] G. E. Hinton, S. Osindero and Y. Teh, "A Fast Learning Algorithm for Deep Belief Nets," in *Neural Computation*, vol. 18, no. 7, pp. 1527-1554, July 2006, doi: 10.1162/neco.2006.18.7.1527.
- [10] D. Pestana et al., "A Full Featured Configurable Accelerator for Object Detection With YOLO," in *IEEE Access*, vol. 9, pp. 75864-75877, 2021, doi: 10.1109/ACCESS.2021.3081818.
- [11] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.
- [12] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2017, pp. 6517-6525, doi: 10.1109/CVPR.2017.690.
- [13] Redmon, J., Farhadi, A. (2018). YOLOv3: An Incremental Improvement. *ArXiv*, abs/1804.02767.
- [14] Bochkovskiy, A., Wang, C., Liao, H.M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. *ArXiv*, abs/2004.10934.
- [15] J. Hu, L. Shen and G. Sun, "Squeeze-and-Excitation Networks," 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 2018, pp. 7132-7141, doi: 10.1109/CVPR.2018.00745.
- [16] Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. 2015. Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks. In *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '15)*. Association for Computing Machinery, New York, NY, USA, 161-170. <https://doi.org/10.1145/2684746.2689060>
- [17] Lin, Tsung-Yi et al. "Microsoft COCO: Common Objects in Context." *European Conference on Computer Vision* (2014).
- [18] Zhewen Yu and Christos-Savvas Bouganis. 2020. A Parameterisable FPGA-Tailored Architecture for YOLOv3-Tiny. In *Applied Reconfigurable Computing. Architectures, Tools, and Applications: 16th International Symposium, ARC 2020, Toledo, Spain, April 1-3, 2020, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 330-344. https://doi.org/10.1007/978-3-030-44534-8_25
- [19] O. Eid and M. A. A. El Ghany, "Hardware Implementation of Yolov4-tiny for Object Detection," 2021 International Conference on Microelectronics (ICM), New Cairo City, Egypt, 2021, pp. 270-275, doi: 10.1109/ICM52667.2021.9664943.